

Let Us C

Fifth Edition

Yashavant P. Kanetkar

Dedicated to baba

Who couldn't be here to see this day...

About the Author

Destiny drew Yashavant Kanetkar towards computers when the IT industry was just making a beginning in India. Having completed his education from VJTI Mumbai and IIT Kanpur in Mechanical Engineering he started his training company in Nagpur.

Yashavant has a passion for writing and is an author of several books in C, C++, VC++, C#, .NET, DirectX and COM programming. He is a much sought after speaker on various technology subjects and is a regular columnist for Express Computers and Developer 2.0. His current affiliations include being a Director of KICIT, a training company and DCube Software Technologies, a software development company. In recognition to his contribution Microsoft awarded him the prestigious ***“Best .NET Technical Contributor”*** award recently. He can be reached at kanetkar@kicit.com.

Preface to the Fifth Edition

It is mid 2004. World has left behind the DOTCOM bust, 9/11 tragedy, the economic downturn, etc. and moved on. Countless Indians have relentlessly worked for close to two decades to successfully establish “India” as a software brand. At times I take secret pleasure in seeing that a book that I have been part of, has contributed in its own little way in shaping so many budding careers that have made the “India” brand acceptable.

Computing and the way people use C for doing it keeps changing as years go by. So overwhelming has been the response to all the previous editions of “Let Us C” that I have now decided that each year I would come up with a new edition of it so that I can keep the readers abreast with the way C is being used at that point in time.

There are two phases in every C programmer’s life. In the first phase he is a learner trying to understand the language elements and their nuances. At this stage he wants a simple learning environment that helps him to master the language. In my opinion, even today there isn’t any learning environment that can beat Turbo C/C++ for simplicity. Hence the first fifteen chapters are written keeping this environment in mind, though a majority of these programs in these chapters would work with any C compiler.

Armed with the knowledge of language elements the C programmer enters the second phase. Here he wishes to use all that he has learnt to create programs that match the ability of programs that he see in today’s world. I am pointing towards programs in Windows and Linux world. Chapters 16 to 21 are devoted to this. I would like to your attention the fact that if you want to program Windows or Linux you need to have a very good grasp over the programming model used by each of these OS. Windows messaging architecture and Linux signaling mechanism are the cases in point. Once you understand these thoroughly rest is just a

matter of time. Chapters 16 to 21 have been written with this motive.

In Linux programming the basic hurdle is in choosing the Linux distribution, compiler, editor, shell, libraries, etc. To get a head-start you can follow the choices that I found most reasonable and simple. They have been mentioned in Chapter 20 and Appendix H. Once you are comfortable you can explore other choices.

In fourth edition of Let Us C there were chapters on ‘Disk Basics’, ‘VDU Basics’, ‘Graphics’, ‘Mouse Programming’, ‘C and Assembly’. Though I used to like these chapters a lot I had to take a decision to drop them since most of them were DOS-centric and would not be so useful in modern-day programming. Modern counterparts of all of these have been covered in Chapters 16 to 21. However, if you still need the chapters from previous edition they are available at www.kicit.com/books/letusc/fourthedition.

Also, all the programs present in the book are available in source code form at www.kicit.com/books/letusc/sourcecode. You are free to download them, improve them, change them, do whatever with them. If you wish to get solutions for the Exercises in the book they are available in another book titled ‘Let Us C Solutions’.

‘Let Us C’ is as much your book as it is mine. So if you feel that I could have done certain job better than what I have, or you have any suggestions about what you would like to see in the next edition, please drop a line to letuscsuggestions@kicit.com.

All the best and happy programming!

Contents

1. Getting Started	1
What is C	2
Getting Started with C	4
The C Character Set	5
Constants, Variables and Keywords	6
Types of C Constants	7
Rules for Constructing Integer Constants	8
Rules for Constructing Real Constants	9
Rules for Constructing Character Constants	10
Types of C Variables	11
Rules for Constructing Variable Names	11
C Keywords	12
The First C Program	13
Compilation and Execution	19
Receiving Input	21
C Instructions	23
Type Declaration Instruction	24
Arithmetic Instruction	25
Integer and Float Conversions	29
Type Conversion in Assignments	29
Hierarchy of Operations	31
Associativity of Operators	34
Control Instructions in C	37
Summary	37
Exercise	38
 2. The Decision Control Structure	 49
Decisions! Decisions!	50
The <i>if</i> Statement	51
The Real Thing	55
Multiple Statements within <i>if</i>	56
The <i>if-else</i> Statement	58

Nested <i>if-elses</i>	61
Forms of <i>if</i>	62
Use of Logical Operators	64
The <i>else if</i> Clause	66
The ! Operator	72
Hierarchy of Operators Revisited	73
A Word of Caution	73
The Conditional Operators	76
Summary	77
Exercise	78
 3. The Loop Control Structure	 97
Loops	98
The <i>while</i> Loop	99
Tips and Traps	101
More Operators	105
The <i>for</i> Loop	107
Nesting of Loops	114
Multiple Initialisations in the <i>for</i> Loop	115
The Odd Loop	116
The <i>break</i> Statement	118
The <i>continue</i> Statement	120
The <i>do-while</i> Loop	121
Summary	124
Exercise	124
 4. The Case Control Structure	 135
Decisions Using <i>switch</i>	136
The Tips and Traps	140
<i>switch</i> Versus <i>if-else</i> Ladder	144
The <i>goto</i> Keyword	145
Summary	148
Exercise	149

5. Functions & Pointers	157
What is a Function	158
Why Use Functions	165
Passing Values between Functions	166
Scope Rule of Functions	171
Calling Convention	172
One Dicey Issue	173
Advanced Features of Functions	174
Function Declaration and Prototypes	175
Call by Value and Call by Reference	178
An Introduction to Pointers	178
Pointer Notation	179
Back to Function Calls	186
Conclusions	189
Recursion	189
Recursion and Stack	194
Adding Functions to the Library	197
Summary	201
Exercise	201
 6. Data Types Revisited	 213
Integers, <i>long</i> and <i>short</i>	214
Integers, <i>signed</i> and <i>unsigned</i>	216
Chars, <i>signed</i> and <i>unsigned</i>	217
Floats and Doubles	219
A Few More Issues...	221
Storage Classes in C	223
Automatic Storage Class	224
Register Storage Class	226
Static Storage Class	227
External Storage Class	230
Which to Use When	233
Summary	234
Exercise	235

7. The C Preprocessor	241
Features of C Preprocessor	242
Macro Expansion	244
Macros with Arguments	248
Macros versus Functions	252
File Inclusion	253
Conditional Compilation	255
<i>#if</i> and <i>#elif</i> Directives	258
Miscellaneous Directives	260
<i>#undef</i> Directive	260
<i>#pragma</i> Directive	261
Summary	263
Exercise	264
 8. Arrays	 269
What are Arrays	270
A Simple Program Using Array	272
More on Arrays	275
Array Initialization	275
Bounds Checking	276
Passing Array Elements to a Function	277
Pointers and Arrays	279
Passing an Entire Array to a Function	286
The Real Thing	287
Two Dimensional Arrays	289
Initializing a 2-Dimensional Array	290
Memory Map of a 2-Dimensional Array	291
Pointers and 2-Dimensional Arrays	292
Pointer to an Array	295
Passing 2-D array to a Function	297
Array of Pointers	300
Three Dimensional Array	302
Summary	304

Exercise	304
9. Puppetting On Strings	327
What are Strings	328
More about Strings	329
Pointers and Strings	334
Standard Library String Functions	335
<i>strlen()</i>	337
<i>strcpy()</i>	339
<i>strcat()</i>	342
<i>strcmp()</i>	343
Two-Dimensional Array of Characters	344
Array of Pointers to Strings	347
Limitation of Array of Pointers to Strings	351
Solution	352
Summary	353
Exercise	354
10. Structures	363
Why Use Structures	364
Declaring a Structure	367
Accessing Structure Elements	370
How Structure Elements are Stored	370
Array of Structures	371
Additional Features of Structures	374
Uses of Structures	383
Summary	384
Exercise	384
11. Console Input/Output	393
Types of I/O	394
Console I/O Functions	395
Formatted Console I/O Functions	396

<i>sprintf()</i> and <i>scanf()</i> Functions	404
Unformatted Console I/O Functions	405
Summary	409
Exercise	409
12. File Input/Output	415
Data Organization	416
File Operations	417
Opening a File	418
Reading from a File	420
Trouble in Opening a File	421
Closing the File	422
Counting Characters, Tabs, Spaces, ...	422
A File-copy Program	424
Writing to a File	425
File Opening Modes	426
String (line) I/O in Files	427
The Awkward Newline	430
Record I/O in Files	430
Text Files and Binary Files	434
Record I/O Revisited	437
Database Management	441
Low Level Disk I/O	447
A Low Level File-copy Program	448
I/O Under Windows	453
Summary	453
Exercise	454
13. More Issues In Input/Output	465
Using <i>argc</i> and <i>argv</i>	466
Detecting Errors in Reading/Writing	470
Standard I/O Devices	472
I/O Redirection	473
Redirecting the Output	474

Redirecting the Input	476
Both Ways at Once	477
Summary	478
Exercise	478
14. Operations On Bits	481
Bitwise Operators	482
One's Complement Operator	484
Right Shift Operator	486
Left Shift Operator	488
Bitwise AND Operator	493
Bitwise OR Operator	498
Bitwise XOR Operator	499
The <i>showbits()</i> Function	500
Summary	501
Exercise	501
15. Miscellaneous Features	505
Enumerated Data Type	506
Uses of Enumerated Data Type	507
Renaming Data Types with <i>typedef</i>	510
Typecasting	511
Bit Fields	513
Pointers to Functions	515
Functions Returning Pointers	518
Functions with Variable Number of Arguments	520
Unions	524
Union of Structures	530
Summary	531
Exercise	531

16. C Under Windows	535
Which Windows...	536
Integers	537
The Use of <i>typedef</i>	537
Pointers in the 32-bit World	539
Memory Management	540
Device Access	543
DOS Programming Model	543
Windows Programming Model	547
Event Driven Model	551
Windows Programming, a Closer Look	552
The First Windows Program	554
Hungarian Notation	558
Summary	558
Exercise	559
 17. Windows Programming	 561
The Role of a Message Box	562
Here Comes the window...	563
More Windows	566
A Real-World Window	567
Creation and Displaying of Window	569
Interaction with Window	570
Reacting to Messages	572
Program Instances	575
Summary	575
Exercise	576
 18. Graphics Under Windows	 579
Graphics as of Now	580
Device Independent Drawing	580

Hello Windows	582
Drawing Shapes	586
Types of Pens	590
Types of Brushes	592
Code and Resources	596
Freehand Drawing, the Paintbrush Style	596
Capturing the Mouse	600
Device Context, a Closer Look	601
Displaying a Bitmap	603
Animation at Work	607
WM_CREATE and <i>OnCreate()</i>	611
WM_TIMER and <i>OnTimer()</i>	611
A Few More Points...	612
Windows, the Endless World...	613
Summary	614
Exercise	615

19. Interaction With Hardware 617

Hardware Interaction	618
Hardware Interaction, DOS Perspective	619
Hardware Interaction, Windows Perspective	623
Communication with Storage Devices	626
The <i>ReadSector()</i> Function	631
Accessing Other Storage Devices	633
Communication with Keyboard	634
Dynamic Linking	635
Windows Hooks	635
Caps Locked, Permanently	637
Did You Press It TTwwiiccee.....	643
Mangling Keys	644
KeyLogger	645
Where is This Leading	646
Summary	647
Exercise	647

20. C Under Linux	649
What is Linux	650
C Programming Under Linux	651
The ‘Hello Linux’ Program	652
Processes	653
Parent and Child Processes	655
More Processes	659
Zombies and Orphans	660
One Interesting Fact	663
Summary	664
Exercise	664
21. More Linux Programming	667
Communication using Signals	668
Handling Multiple Signals	671
Registering a Common Handler	673
Blocking Signals	675
Event Driven Programming	678
Where Do You Go From Here	684
Summary	684
Exercise	685
<i>Appendix A – Precedence Table</i>	687
<i>Appendix B – Standard Library Functions</i>	691
<i>Appendix C – Chasing the Bugs</i>	701
<i>Appendix D – Hexadecimal Numbering</i>	713
<i>Appendix E – ASCII Chart</i>	719
<i>Appendix F – Helper.h File</i>	725
<i>Appendix G – Boot Parameters</i>	729
<i>Appendix H – Linux Installation</i>	735
<i>Index</i>	739

1 Getting Started

- What is C
- Getting Started with C
 - The C Character Set
 - Constants, Variables and Keywords
 - Types of C Constants
 - Rules for Constructing Integer Constants
 - Rules for Constructing Real Constants
 - Rules for Constructing Character Constants
 - Types of C Variables
 - Rules for Constructing Variable Names
 - C Keywords
- The First C Program
- Compilation and Execution
- Receiving Input
- C Instructions
 - Type Declaration Instruction
 - Arithmetic Instruction
 - Integer and Float Conversions
 - Hierarchy of Operations
 - Associativity Of Operators
- Control Instruction in C
- Summary
- Exercise

Before we can begin to write serious programs in C, it would be interesting to find out what really is C, how it came into existence and how does it compare with other computer languages. In this chapter we would briefly outline these issues.

Four important aspects of any language are the way it stores data, the way it operates upon this data, how it accomplishes input and output and how it lets you control the sequence of execution of instructions in a program. We would discuss the first three of these building blocks in this chapter.

What is C

C is a programming language developed at AT & T's Bell Laboratories of USA in 1972. It was designed and written by a man named Dennis Ritchie. In the late seventies C began to replace the more familiar languages of that time like PL/I, ALGOL, etc. No one pushed C. It wasn't made the 'official' Bell Labs language. Thus, without any advertisement C's reputation spread and its pool of users grew. Ritchie seems to have been rather surprised that so many programmers preferred C to older languages like FORTRAN or PL/I, or the newer ones like Pascal and APL. But, that's what happened.

Possibly why C seems so popular is because it is reliable, simple and easy to use. Moreover, in an industry where newer languages, tools and technologies emerge and vanish day in and day out, a language that has survived for more than 3 decades has to be really good.

An opinion that is often heard today is – “C has been already superceded by languages like C++, C# and Java, so why bother to

learn C today”. I seriously beg to differ with this opinion. There are several reasons for this:

- (a) I believe that nobody can learn C++ or Java directly. This is because while learning these languages you have things like classes, objects, inheritance, polymorphism, templates, exception handling, references, etc. do deal with apart from knowing the actual language elements. Learning these complicated concepts when you are not even comfortable with the basic language elements is like putting the cart before the horse. Hence one should first learn all the language elements very thoroughly using C language before migrating to C++, C# or Java. Though this two step learning process may take more time, but at the end of it you will definitely find it worth the trouble.
- (b) C++, C# or Java make use of a principle called Object Oriented Programming (OOP) to organize the program. This organizing principle has lots of advantages to offer. But even while using this organizing principle you would still need a good hold over the language elements of C and the basic programming skills.
- (c) Though many C++ and Java based programming tools and frameworks have evolved over the years the importance of C is still unchallenged because knowingly or unknowingly while using these frameworks and tools you would be still required to use the core C language elements—another good reason why one should learn C before C++, C# or Java.
- (d) Major parts of popular operating systems like Windows, UNIX, Linux is still written in C. This is because even today when it comes to performance (speed of execution) nothing beats C. Moreover, if one is to extend the operating system to work with new devices one needs to write device driver programs. These programs are exclusively written in C.

- (e) Mobile devices like cellular phones and palmtops are becoming increasingly popular. Also, common consumer devices like microwave oven, washing machines and digital cameras are getting smarter by the day. This smartness comes from a microprocessor, an operating system and a program embedded in these devices. These programs not only have to run fast but also have to work in a limited amount of memory. No wonder that such programs are written in C. With these constraints on time and space, C is the language of choice while building such operating systems and programs.
- (f) You must have seen several professional 3D computer games where the user navigates some object, like say a spaceship and fires bullets at the invaders. The essence of all such games is speed. Needless to say, such games won't become popular if they take a long time to move the spaceship or to fire a bullet. To match the expectations of the player the game has to react fast to the user inputs. This is where C language scores over other languages. Many popular gaming frameworks have been built using C language.
- (g) At times one is required to very closely interact with the hardware devices. Since C provides several language elements that make this interaction feasible without compromising the performance it is the preferred choice of the programmer.

I hope that these are very convincing reasons why one should adopt C as the first and the very important step in your quest for learning programming languages.

Getting Started with C

Communicating with a computer involves speaking the language the computer understands, which immediately rules out English as the language of communication with computer. However, there is

a close analogy between learning English language and learning C language. The classical method of learning English is to first learn the alphabets used in the language, then learn to combine these alphabets to form words, which in turn are combined to form sentences and sentences are combined to form paragraphs. Learning C is similar and easier. Instead of straight-away learning how to write programs, we must first know what alphabets, numbers and special symbols are used in C, then how using them constants, variables and keywords are constructed, and finally how are these combined to form an instruction. A group of instructions would be combined later on to form a program. This is illustrated in the Figure 1.1.

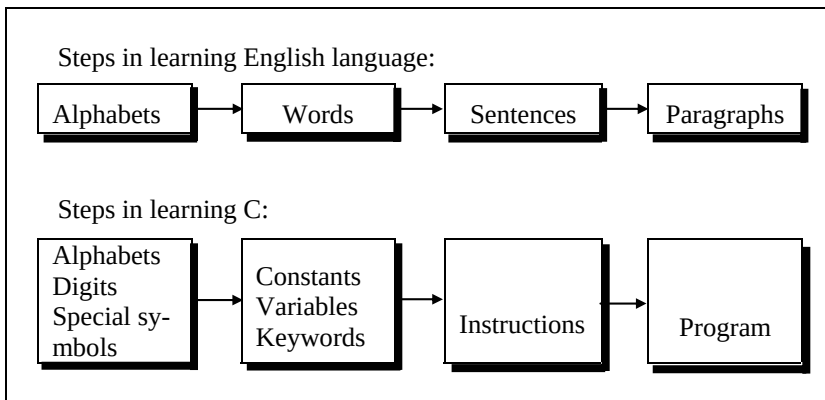


Figure 1.1

The C Character Set

A character denotes any alphabet, digit or special symbol used to represent information. Figure 1.2 shows the valid alphabets, numbers and special symbols allowed in C.

Alphabets	A, B,, Y, Z a, b,, y, z
Digits	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Special symbols	~ ‘ ! @ # % ^ & * () _ - + = \ { } [] : ; " ' < > , . ? /

Figure 1.2

Constants, Variables and Keywords

The alphabets, numbers and special symbols when properly combined form constants, variables and keywords. Let us see what are ‘constants’ and ‘variables’ in C. A constant is an entity that doesn’t change whereas a variable is an entity that may change.

In any program we typically do lots of calculations. The results of these calculations are stored in computers memory. Like human memory the computer memory also consists of millions of cells. The calculated values are stored in these memory cells. To make the retrieval and usage of these values easy these memory cells (also called memory locations) are given names. Since the value stored in each location may change the names given to these locations are called variable names. Consider the following example.

Here 3 is stored in a memory location and a name **x** is given to it. Then we are assigning a new value 5 to the same memory location **x**. This would overwrite the earlier value 3, since a memory location can hold only one value at a time. This is shown in Figure 1.3.

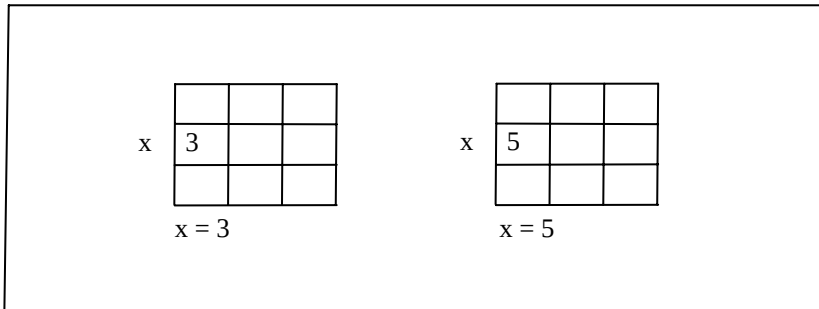


Figure 1.3

Since the location whose name is **x** can hold different values at different times **x** is known as a variable. As against this, 3 or 5 do not change, hence are known as constants.

Types of C Constants

C constants can be divided into two major categories:

- (a) Primary Constants
- (b) Secondary Constants

These constants are further categorized as shown in Figure 1.4.

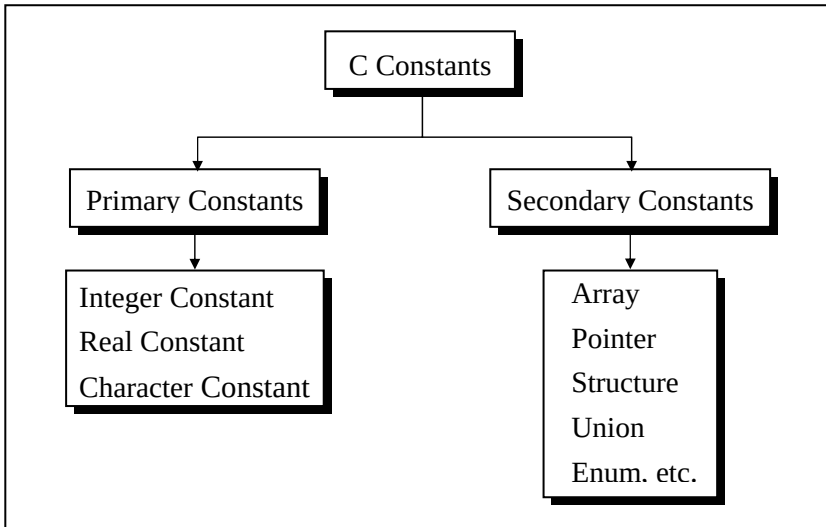


Figure 1.4

At this stage we would restrict our discussion to only Primary Constants, namely, Integer, Real and Character constants. Let us see the details of each of these constants. For constructing these different types of constants certain rules have been laid down. These rules are as under:

Rules for Constructing Integer Constants

- (a) An integer constant must have at least one digit.
- (b) It must not have a decimal point.
- (c) It can be either positive or negative.
- (d) If no sign precedes an integer constant it is assumed to be positive.
- (e) No commas or blanks are allowed within an integer constant.
- (f) The allowable range for integer constants is -32768 to 32767.

Truly speaking the range of an Integer constant depends upon the compiler. For a 16-bit compiler like Turbo C or Turbo C++ the

range is -32768 to 32767 . For a 32-bit compiler the range would be even greater. Question like what exactly do you mean by a 16-bit or a 32-bit compiler, what range of an Integer constant has to do with the type of compiler and such questions are discussed in detail in Chapter 16. Till that time it would be assumed that we are working with a 16-bit compiler.

Ex.: 426
+782
-8000
-7605

Rules for Constructing Real Constants

Real constants are often called Floating Point constants. The real constants could be written in two forms—Fractional form and Exponential form.

Following rules must be observed while constructing real constants expressed in fractional form:

- (a) A real constant must have at least one digit.
- (b) It must have a decimal point.
- (c) It could be either positive or negative.
- (d) Default sign is positive.
- (e) No commas or blanks are allowed within a real constant.

Ex.: +325.34
426.0
-32.76
-48.5792

The exponential form of representation of real constants is usually used if the value of the constant is either too small or too large. It however doesn't restrict us in any way from using exponential form of representation for other real constants.

In exponential form of representation, the real constant is represented in two parts. The part appearing before 'e' is called mantissa, whereas the part following 'e' is called exponent.

Following rules must be observed while constructing real constants expressed in exponential form:

- (a) The mantissa part and the exponential part should be separated by a letter e.
- (b) The mantissa part may have a positive or negative sign.
- (c) Default sign of mantissa part is positive.
- (d) The exponent must have at least one digit, which must be a positive or negative integer. Default sign is positive.
- (e) Range of real constants expressed in exponential form is $-3.4e38$ to $3.4e38$.

Ex.: +3.2e-5
4.1e8
-0.2e+3
-3.2e-5

Rules for Constructing Character Constants

- (a) A character constant is a single alphabet, a single digit or a single special symbol enclosed within single inverted commas. Both the inverted commas should point to the left. For example, 'A' is a valid character constant whereas 'A' is not.
- (b) The maximum length of a character constant can be 1 character.

Ex.: 'A'
'l'
'5'
'='

Types of C Variables

As we saw earlier, an entity that may vary during program execution is called a variable. Variable names are names given to locations in memory. These locations can contain integer, real or character constants. In any language, the types of variables that it can support depend on the types of constants that it can handle. This is because a particular type of variable can hold only the same type of constant. For example, an integer variable can hold only an integer constant, a real variable can hold only a real constant and a character variable can hold only a character constant.

The rules for constructing different types of constants are different. However, for constructing variable names of all types the same set of rules apply. These rules are given below.

Rules for Constructing Variable Names

- (a) A variable name is any combination of 1 to 31 alphabets, digits or underscores. Some compilers allow variable names whose length could be up to 247 characters. Still, it would be safer to stick to the rule of 31 characters. Do not create unnecessarily long variable names as it adds to your typing effort.
- (b) The first character in the variable name must be an alphabet or underscore.
- (c) No commas or blanks are allowed within a variable name.
- (d) No special symbol other than an underscore (as in **gross_sal**) can be used in a variable name.

Ex.: `si_int`
 `m_hra`
 `pop_e_89`

These rules remain same for all the types of primary and secondary variables. Naturally, the question follows... how is C able to differentiate between these variables? This is a rather simple

matter. C compiler is able to distinguish between the variable names by making it compulsory for you to declare the type of any variable name that you wish to use in a program. This type declaration is done at the beginning of the program. Following are the examples of type declaration statements:

```
Ex.: int si, m_hra ;  
      float bassal ;  
      char code ;
```

Since, the maximum allowable length of a variable name is 31 characters, an enormous number of variable names can be constructed using the above-mentioned rules. It is a good practice to exploit this enormous choice in naming variables by using meaningful variable names.

Thus, if we want to calculate simple interest, it is always advisable to construct meaningful variable names like **prin**, **roi**, **noy** to represent Principle, Rate of interest and Number of years rather than using the variables **a**, **b**, **c**.

C Keywords

Keywords are the words whose meaning has already been explained to the C compiler (or in a broad sense to the computer). The keywords **cannot** be used as variable names because if we do so we are trying to assign a new meaning to the keyword, which is not allowed by the computer. Some C compilers allow you to construct variable names that exactly resemble the keywords. However, it would be safer not to mix up the variable names and the keywords. The keywords are also called ‘Reserved words’.

There are only 32 keywords available in C. Figure 1.5 gives a list of these keywords for your ready reference. A detailed discussion of each of these keywords would be taken up in later chapters wherever their use is relevant.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Figure 1.5

Note that compiler vendors (like Microsoft, Borland, etc.) provide their own keywords apart from the ones mentioned above. These include extended keywords like **near**, **far**, **asm**, etc. Though it has been suggested by the ANSI committee that every such compiler specific keyword should be preceded by two underscores (as in **__asm**), not every vendor follows this rule.

The First C Program

Armed with the knowledge about the types of variables, constants & keywords the next logical step is to combine them to form instructions. However, instead of this, we would write our first C program now. Once we have done that we would see in detail the instructions that it made use of.

Before we begin with our first C program do remember the following rules that are applicable to all C programs:

- (a) Each instruction in a C program is written as a separate statement. Therefore a complete C program would comprise of a series of statements.

- (b) The statements in a program must appear in the same order in which we wish them to be executed; unless of course the logic of the problem demands a deliberate ‘jump’ or transfer of control to a statement, which is out of sequence.
- (c) Blank spaces may be inserted between two words to improve the readability of the statement. However, no blank spaces are allowed within a variable, constant or keyword.
- (d) All statements are entered in small case letters.
- (e) C has no specific rules for the position at which a statement is to be written. That’s why it is often called a free-form language.
- (f) Every C statement must end with a `;`. Thus `;` acts as a statement terminator.

Let us now write down our first C program. It would simply calculate simple interest for a set of values representing principle, number of years and rate of interest.

```
/* Calculation of simple interest */
/* Author gekay Date: 25/05/2004 */
main( )
{
    int  p, n ;
    float r, si ;

    p = 1000 ;
    n = 3 ;
    r = 8.5 ;

    /* formula for simple interest */
    si = p * n * r / 100 ;

    printf ( "%f" , si ) ;
```

```
}
```

Now a few useful tips about the program...

- Comment about the program should be enclosed within `/* */`. For example, the first two statements in our program are comments.
- Though comments are not necessary, it is a good practice to begin a program with a comment indicating the purpose of the program, its author and the date on which the program was written.
- Any number of comments can be written at any place in the program. For example, a comment can be written before the statement, after the statement or within the statement as shown below:

```
/* formula */ si = p * n * r / 100 ;  
si = p * n * r / 100 ; /* formula */  
si = p * n * r /* formula */ 100 ;
```

- Sometimes it is not so obvious as to what a particular statement in a program accomplishes. At such times it is worthwhile mentioning the purpose of the statement (or a set of statements) using a comment. For example:

```
/* formula for simple interest */  
si = p * n * r / 100 ;
```

- Often programmers seem to ignore writing of comments. But when a team is building big software well commented code is almost essential for other team members to understand it.

- Although a lot of comments are probably not necessary in this program, it is usually the case that programmers tend to use too few comments rather than too many. An adequate number of comments can save hours of misery and suffering when you later try to figure out what the program does.
- The normal language rules do not apply to text written within `/* .. */`. Thus we can type this text in small case, capital or a combination. This is because the comments are solely given for the understanding of the programmer or the fellow programmers and are completely ignored by the compiler.

- Comments cannot be nested. For example,

```
/* Cal of SI /* Author sam date 01/01/2002 */ */
```

is invalid.

- A comment can be split over more than one line, as in,

```
/* This is  
   a jazzy  
   comment */
```

Such a comment is often called a multi-line comment.

- **main()** is a collective name given to a set of statements. This name has to be **main()**, it cannot be anything else. All statements that belong to **main()** are enclosed within a pair of braces { } as shown below.

```
main()  
{  
    statement 1 ;  
    statement 2 ;
```

```
    statement 3 ;  
}
```

- Technically speaking **main()** is a function. Every function has a pair of parentheses () associated with it. We would discuss functions and their working in great detail in Chapter 5.
- Any variable used in the program must be declared before using it. For example,

```
int  p, n ;  
float r, si ;
```

- Any C statement always ends with a ;

For example,

```
float r, si ;  
r = 8.5 ;
```

- In the statement,

```
si = p * n * r / 100 ;
```

* and / are the arithmetic operators. The arithmetic operators available in C are +, -, * and /. C is very rich in operators. There are about 45 operators available in C. Surprisingly there is no operator for exponentiation... a slip, which can be forgiven considering the fact that C has been developed by an individual, not by a committee.

- Once the value of **si** is calculated it needs to be displayed on the screen. Unlike other languages, C does not contain any instruction to display output on the screen. All output to screen is achieved using readymade library functions. One such

function is **printf()**. We have used it display on the screen the value contained in **si**.

The general form of **printf()** function is,

```
printf ( "<format string>", <list of variables> );
```

<format string> can contain,

%f for printing real values

%d for printing integer values

%c for printing character values

In addition to format specifiers like **%f**, **%d** and **%c** the format string may also contain any other characters. These characters are printed as they are when the **printf()** is executed.

Following are some examples of usage of **printf()** function:

```
printf ( "%f", si );  
printf ( "%d %d %f %f", p, n, r, si );  
printf ( "Simple interest = Rs. %f", si );  
printf ( "Prin = %d \nRate = %f", p, r );
```

The output of the last statement would look like this...

```
Prin = 1000
```

```
Rate = 8.5
```

What is ‘\n’ doing in this statement? It is called newline and it takes the cursor to the next line. Therefore, you get the output split over two lines. ‘\n’ is one of the several Escape Sequences available in C. These are discussed in detail in Chapter 11. Right now, all that we can say is ‘\n’ comes in

handy when we want to format the output properly on separate lines.

printf() can not only print values of variables, it can also print the result of an expression. An expression is nothing but a valid combination of constants, variables and operators. Thus, 3, 3 + 2, c and a + b * c - d all are valid expressions. The results of these expressions can be printed as shown below:

```
printf ( "%d %d %d %d", 3, 3 + 2, c, a + b * c - d );
```

Note that **3** and **c** also represent valid expressions.

Compilation and Execution

Once you have written the program you need to type it and instruct the machine to execute it. To type your C program you need another program called Editor. Once the program has been typed it needs to be converted to machine language (0s and 1s) before the machine can execute it. To carry out this conversion we need another program called Compiler. Compiler vendors provide an Integrated Development Environment (IDE) which consists of an Editor as well as the Compiler.

There are several such IDEs available in the market targeted towards different operating systems. For example, Turbo C, Turbo C++ and Microsoft C are some of the popular compilers that work under MS-DOS; Visual C++ and Borland C++ are the compilers that work under Windows, whereas gcc compiler works under Linux. Note that Turbo C++, Microsoft C++ and Borland C++ software also contain a C compiler bundled with them. If you are a beginner you would be better off using a simple compiler like Turbo C or Turbo C++. Once you have mastered the language elements you can then switch over to more sophisticated compilers like Visual C++ under Windows or gcc under Linux. Most of the

programs in this book would work with all the compilers. Wherever there is a deviation I would point it out that time.

Assuming that you are using a Turbo C or Turbo C++ compiler here are the steps that you need to follow to compile and execute your first C program...

- (a) Start the compiler at **C>** prompt. The compiler (TC.EXE is usually present in **C:\TC\BIN** directory).
- (b) Select **New** from the **File** menu.
- (c) Type the program.
- (d) Save the program using **F2** under a proper name (say Program1.c).
- (e) Use **Ctrl + F9** to compile and execute the program.
- (f) Use **Alt + F5** to view the output.

Note that on compiling the program its machine language equivalent is stored as an EXE file (Program1.EXE) on the disk. This file is called an executable file. If we copy this file to another machine we can execute it there without being required to recompile it. In fact the other machine need not even have a compiler to be able to execute the file.

A word of caution! If you run this program in Turbo C++ compiler, you may get an error — “The function printf should have a prototype”. To get rid of this error, perform the following steps and then recompile the program.

- (a) Select ‘Options’ menu and then select ‘Compiler | C++ Options’. In the dialog box that pops up, select ‘CPP always’ in the ‘Use C++ Compiler’ options.
- (b) Again select ‘Options’ menu and then select ‘Environment | Editor’. Make sure that the default extension is ‘C’ rather than ‘CPP’.

Receiving Input

In the program discussed above we assumed the values of **p**, **n** and **r** to be 1000, 3 and 8.5. Every time we run the program we would get the same value for simple interest. If we want to calculate simple interest for some other set of values then we are required to make the relevant change in the program, and again compile and execute it. Thus the program is not general enough to calculate simple interest for any set of values without being required to make a change in the program. Moreover, if you distribute the EXE file of this program to somebody he would not even be able to make changes in the program. Hence it is a good practice to create a program that is general enough to work for any set of values.

To make the program general the program itself should ask the user to supply the values of **p**, **n** and **r** through the keyboard during execution. This can be achieved using a function called **scanf()**. This function is a counter-part of the **printf()** function. **printf()** outputs the values to the screen whereas **scanf()** receives them from the keyboard. This is illustrated in the program shown below.

```
/* Calculation of simple interest */
/* Author gekay Date 25/05/2004 */
main()
{
    int  p, n ;
    float r, si ;
    printf ( "Enter values of p, n, r" ) ;
    scanf ( "%d %d %f", &p, &n, &r ) ;

    si = p * n * r / 100 ;
    printf ( "%f", si ) ;
}
```

The first **printf()** outputs the message ‘Enter values of p, n, r’ on the screen. Here we have not used any expression in **printf()** which means that using expressions in **printf()** is optional.

Note that the ampersand (&) before the variables in the **scanf()** function is a must. & is an ‘Address of’ operator. It gives the location number used by the variable in memory. When we say **&a**, we are telling **scanf()** at which memory location should it store the value supplied by the user from the keyboard. The detailed working of the & operator would be taken up in Chapter 5.

Note that a blank, a tab or a new line must separate the values supplied to **scanf()**. Note that a blank is creating using a spacebar, tab using the Tab key and new line using the Enter key. This is shown below:

Ex.: The three values separated by blank

1000 5 15.5

Ex.: The three values separated by tab.

1000 5 15.5

Ex.: The three values separated by newline.

1000
5
15.5

So much for the tips. How about another program to give you a feel of things...

```
/* Just for fun. Author: Bozo */  
main()  
{  
    int num ;  
  
    printf ( "Enter a number" );
```

```
scanf ( "%d", &num ) ;

printf ( "Now I am letting you on a secret..." ) ;
printf ( "You have just entered the number %d", num ) ;
}
```

C Instructions

Now that we have written a few programs let us look at the instructions that we used in these programs. There are basically three types of instructions in C:

- (a) Type Declaration Instruction
- (b) Arithmetic Instruction
- (c) Control Instruction

The purpose of each of these instructions is given below:

- (a) Type declaration instruction – To declare the type of variables used in a C program.
- (b) Arithmetic instruction – To perform arithmetic operations between constants and variables.
- (c) Control instruction – To control the sequence of execution of various statements in a C program.

Since, the elementary C programs would usually contain only the type declaration and the arithmetic instructions; we would discuss only these two instructions at this stage. The other types of instructions would be discussed in detail in the subsequent chapters.

Type Declaration Instruction

This instruction is used to declare the type of variables being used in the program. Any variable used in the program must be declared before using it in any statement. The type declaration statement is written at the beginning of **main()** function.

Ex.: `int bas ;`
`float rs, grosssal ;`
`char name, code ;`

There are several subtle variations of the type declaration instruction. These are discussed below:

- (a) While declaring the type of variable we can also initialize it as shown below.

```
int i = 10, j = 25 ;  
float a = 1.5, b = 1.99 + 2.4 * 1.44 ;
```

- (b) The order in which we define the variables is sometimes important sometimes not. For example,

```
int i = 10, j = 25 ;
```

is same as

```
int j = 25, i = 10 ;
```

However,

```
float a = 1.5, b = a + 3.1 ;
```

is alright, but

```
float b = a + 3.1, a = 1.5 ;
```

is not. This is because here we are trying to use **a** even before defining it.

- (c) The following statements would work

```
int a, b, c, d ;  
a = b = c = 10 ;
```

However, the following statement would not work

```
int a = b = c = d = 10 ;
```

Once again we are trying to use **b** (to assign to **a**) before defining it.

Arithmetic Instruction

A C arithmetic instruction consists of a variable name on the left hand side of = and variable names & constants on the right hand side of =. The variables and constants appearing on the right hand side of = are connected by arithmetic operators like +, -, *, and /.

```
Ex.: int ad ;  
      float kot, deta, alpha, beta, gamma ;  
      ad = 3200 ;  
      kot = 0.0056 ;  
      deta = alpha * beta / gamma + 3.2 * 2 / 5 ;
```

Here,

*****, **/**, **-**, **+** are the arithmetic operators.

= is the assignment operator.

2, 5 and 3200 are integer constants.

3.2 and 0.0056 are real constants.

ad is an integer variable.

kot, deta, alpha, beta, gamma are real variables.

The variables and constants together are called ‘operands’ that are operated upon by the ‘arithmetic operators’ and the result is assigned, using the assignment operator, to the variable on left-hand side.

A C arithmetic statement could be of three types. These are as follows:

- (a) Integer mode arithmetic statement - This is an arithmetic statement in which all operands are either integer variables or integer constants.

```
Ex.: int i, king, issac, noteit ;  
      i = i + 1 ;  
      king = issac * 234 + noteit - 7689 ;
```

- (b) Real mode arithmetic statement - This is an arithmetic statement in which all operands are either real constants or real variables.

```
Ex.: float qbee, antink, si, prin, anoy, roi ;  
      qbee = antink + 23.123 / 4.5 * 0.3442 ;  
      si = prin * anoy * roi / 100.0 ;
```

- (c) Mixed mode arithmetic statement - This is an arithmetic statement in which some of the operands are integers and some of the operands are real.

```
Ex.: float si, prin, anoy, roi, avg ;  
      int a, b, c, num ;  
      si = prin * anoy * roi / 100.0 ;  
      avg = ( a + b + c + num ) / 4 ;
```

It is very important to understand how the execution of an arithmetic statement takes place. Firstly, the right hand side is evaluated using constants and the numerical values stored in the variable names. This value is then assigned to the variable on the left-hand side.

Though Arithmetic instructions look simple to use one often commits mistakes in writing them. Let us take a closer look at these statements. Note the following points carefully.

- (a) C allows only one variable on left-hand side of $=$. That is, $z = k * l$ is legal, whereas $k * l = z$ is illegal.
- (b) In addition to the division operator C also provides a modular division operator. This operator returns the remainder on dividing one integer with another. Thus the expression $10 / 2$ yields 5, whereas, $10 \% 2$ yields 0. Note that the modulus operator ($\%$) cannot be applied on a float. Also note that on using $\%$ the sign of the remainder is always same as the sign of the numerator. Thus $-5 \% 2$ yields -1 , whereas, $5 \% -2$ yields 1.
- (c) An arithmetic instruction is often used for storing character constants in character variables.

```
char a, b, d ;  
a = 'F' ;  
b = 'G' ;  
d = '+' ;
```

When we do this the ASCII values of the characters are stored in the variables. ASCII values are used to represent any character in memory. The ASCII values of 'F' and 'G' are 70 and 71 (refer the ASCII Table in Appendix E).

- (d) Arithmetic operations can be performed on **ints**, **floats** and **chars**.

Thus the statements,

```
char x, y ;  
int z ;  
x = 'a' ;  
y = 'b' ;  
z = x + y ;
```

are perfectly valid, since the addition is performed on the ASCII values of the characters and not on characters themselves. The ASCII values of 'a' and 'b' are 97 and 98, and hence can definitely be added.

- (e) No operator is assumed to be present. It must be written explicitly. In the following example, the multiplication operator after b must be explicitly written.

<code>a = c.d.b(xy)</code>	usual arithmetic statement
<code>b = c * d * b * (x * y)</code>	C statement

- (f) Unlike other high level languages, there is no operator for performing exponentiation operation. Thus following statements are invalid.

```
a = 3 ** 2 ;  
b = 3 ^ 2 ;
```

If we want to do the exponentiation we can get it done this way:

```
#include <math.h>  
main()  
{  
    int a ;  
    a = pow ( 3, 2 ) ;  
    printf ( "%d", a ) ;  
}
```

Here **pow()** function is a standard library function. It is being used to raise 3 to the power of 2. **#include <math.h>** is a preprocessor directive. It is being used here to ensure that the **pow()** function works correctly. We would learn more about standard library functions in Chapter 5 and about preprocessor in Chapter 7.

Integer and Float Conversions

In order to effectively develop C programs, it will be necessary to understand the rules that are used for the implicit conversion of floating point and integer values in C. These are mentioned below. Note them carefully.

- (a) An arithmetic operation between an integer and integer always yields an integer result.
- (b) An operation between a real and real always yields a real result.
- (c) An operation between an integer and real always yields a real result. In this operation the integer is first promoted to a real and then the operation is performed. Hence the result is real.

I think a few practical examples shown in the following figure would put the issue beyond doubt.

Operation	Result	Operation	Result
5 / 2	2	2 / 5	0
5.0 / 2	2.5	2.0 / 5	0.4
5 / 2.0	2.5	2 / 5.0	0.4
5.0 / 2.0	2.5	2.0 / 5.0	0.4

Figure 1.6

Type Conversion in Assignments

It may so happen that the type of the expression and the type of the variable on the left-hand side of the assignment operator may not be same. In such a case the value of the expression is promoted or

demoted depending on the type of the variable on left-hand side of `=`.

For example, consider the following assignment statements.

```
int i;  
float b;  
i = 3.5;  
b = 30;
```

Here in the first assignment statement though the expression's value is a **float** (3.5) it cannot be stored in **i** since it is an **int**. In such a case the **float** is demoted to an **int** and then its value is stored. Hence what gets stored in **i** is 3. Exactly opposite happens in the next statement. Here, 30 is promoted to 30.000000 and then stored in **b**, since **b** being a **float** variable cannot hold anything except a **float** value.

Instead of a simple expression used in the above examples if a complex expression occurs, still the same rules apply. For example, consider the following program fragment.

```
float a, b, c;  
int s;  
s = a * b * c / 100 + 32 / 4 - 3 * 1.1;
```

Here, in the assignment statement some operands are **ints** whereas others are **floats**. As we know, during evaluation of the expression the **ints** would be promoted to **floats** and the result of the expression would be a **float**. But when this **float** value is assigned to **s** it is again demoted to an **int** and then stored in **s**.

Observe the results of the arithmetic statements shown in Figure 1.7. It has been assumed that **k** is an integer variable and **a** is a real variable.

Arithmetic Instruction	Result	Arithmetic Instruction	Result
<code>k = 2 / 9</code>	0	<code>a = 2 / 9</code>	0.0
<code>k = 2.0 / 9</code>	0	<code>a = 2.0 / 9</code>	0.2222
<code>k = 2 / 9.0</code>	0	<code>a = 2 / 9.0</code>	0.2222
<code>k = 2.0 / 9.0</code>	0	<code>a = 2.0 / 9.0</code>	0.2222
<code>k = 9 / 2</code>	4	<code>a = 9 / 2</code>	4.0
<code>k = 9.0 / 2</code>	4	<code>a = 9.0 / 2</code>	4.5
<code>k = 9 / 2.0</code>	4	<code>a = 9 / 2.0</code>	4.5
<code>k = 9.0 / 2.0</code>	4	<code>a = 9.0 / 2.0</code>	4.5

Figure 1.7

Note that though the following statements give the same result, 0, the results are obtained differently.

```
k = 2 / 9 ;
k = 2.0 / 9 ;
```

In the first statement, since both 2 and 9 are integers, the result is an integer, i.e. 0. This 0 is then assigned to **k**. In the second statement 9 is promoted to 9.0 and then the division is performed. Division yields 0.222222. However, this cannot be stored in **k**, **k** being an **int**. Hence it gets demoted to 0 and then stored in **k**.

Hierarchy of Operations

While executing an arithmetic statement, which has two or more operators, we may have some problems as to how exactly does it get executed. For example, does the expression `2 * x - 3 * y` correspond to $(2x)-(3y)$ or to $2(x-3y)$? Similarly, does `A / B * C` correspond to $A / (B * C)$ or to $(A / B) * C$? To answer these questions satisfactorily one has to understand the ‘hierarchy’ of operations. The priority or precedence in which the operations in

an arithmetic statement are performed is called the hierarchy of operations. The hierarchy of commonly used operators is shown in Figure 1.8.

Priority	Operators	Description
1 st	* / %	multiplication, division, modular division
2 nd	+ -	addition, subtraction
3 rd	=	assignment

Figure 1.8

Now a few tips about usage of operators in general.

- (a) Within parentheses the same hierarchy as mentioned in Figure 1.11 is operative. Also, if there are more than one set of parentheses, the operations within the innermost parentheses would be performed first, followed by the operations within the second innermost pair and so on.
- (b) We must always remember to use pairs of parentheses. A careless imbalance of the right and left parentheses is a common error. Best way to avoid this error is to type () and then type an expression inside it.

A few examples would clarify the issue further.

Example 1.1: Determine the hierarchy of operations and evaluate the following expression:

$$i = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8$$

Stepwise evaluation of this expression is shown below:

$$i = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8$$

$i = 6 / 4 + 4 / 4 + 8 - 2 + 5 / 8$	operation: *
$i = 1 + 4 / 4 + 8 - 2 + 5 / 8$	operation: /
$i = 1 + 1 + 8 - 2 + 5 / 8$	operation: /
$i = 1 + 1 + 8 - 2 + 0$	operation: /
$i = 2 + 8 - 2 + 0$	operation: +
$i = 10 - 2 + 0$	operation: +
$i = 8 + 0$	operation: -
$i = 8$	operation: +

Note that $6 / 4$ gives 1 and not 1.5. This so happens because 6 and 4 both are integers and therefore would evaluate to only an integer constant. Similarly $5 / 8$ evaluates to zero, since 5 and 8 are integer constants and hence must return an integer value.

Example 1.2: Determine the hierarchy of operations and evaluate the following expression:

$$kk = 3 / 2 * 4 + 3 / 8 + 3$$

Stepwise evaluation of this expression is shown below:

$kk = 3 / 2 * 4 + 3 / 8 + 3$	
$kk = 1 * 4 + 3 / 8 + 3$	operation: /
$kk = 4 + 3 / 8 + 3$	operation: *
$kk = 4 + 0 + 3$	operation: /
$kk = 4 + 3$	operation: +
$kk = 7$	operation: +

Note that $3 / 8$ gives zero, again for the same reason mentioned in the previous example.

All operators in C are ranked according to their precedence. And mind you there are as many as 45 odd operators in C, and these can affect the evaluation of an expression in subtle and unexpected ways if we aren't careful. Unfortunately, there are no simple rules that one can follow, such as “BODMAS” that tells algebra students in which order does an expression evaluate. We have not

encountered many out of these 45 operators, so we won't pursue the subject of precedence any further here. However, it can be realized at this stage that it would be almost impossible to remember the precedence of all these operators. So a full-fledged list of all operators and their precedence is given in Appendix A. This may sound daunting, but when its contents are absorbed in small bites, it becomes more palatable.

So far we have seen how the computer evaluates an arithmetic statement written in C. But our knowledge would be incomplete unless we know how to convert a general arithmetic statement to a C statement. C can handle any complex expression with ease. Some of the examples of C expressions are shown in Figure 1.9.

Algebraic Expression	C Expression
$a \times b - c \times d$	<code>a * b - c * d</code>
$(m + n)(a + b)$	<code>(m + n) * (a + b)</code>
$3x^2 + 2x + 5$	<code>3 * x * x + 2 * x + 5</code>
$\frac{a + b + c}{d + e}$	<code>(a + b + c) / (d + e)</code>
$\left[\frac{2BY}{d+1} - \frac{x}{3(z+y)} \right]$	<code>2 * b * y / (d + 1) - x / 3 * (z + y)</code>

Figure 1.9

Associativity of Operators

When an expression contains two operators of equal priority the tie between them is settled using the associativity of the operators. Associativity can be of two types—Left to Right or Right to Left. Left to Right associativity means that the left operand must be

unambiguous. Unambiguous in what sense? It must not be involved in evaluation of any other sub-expression. Similarly, in case of Right to Left associativity the right operand must be unambiguous. Let us understand this with an example.

Consider the expression

$$a = 3 / 2 * 5 ;$$

Here there is a tie between operators of same priority, that is between / and *. This tie is settled using the associativity of / and *. But both enjoy Left to Right associativity. Figure 1.10 shows for each operator which operand is unambiguous and which is not.

Operator	Left	Right	Remark
/	3	2 or 2 * 5	Left operand is unambiguous, Right is not
*	3 / 2 or 2	5	Right operand is unambiguous, Left is not

Figure 1.10

Since both / and * have L to R associativity and only / has unambiguous left operand (necessary condition for L to R associativity) it is performed earlier.

Consider one more expression

$$a = b = 3 ;$$

Here both assignment operators have the same priority and same associativity (Right to Left). Figure 1.11 shows for each operator which operand is unambiguous and which is not.

Operator	Left	Right	Remark
=	a	b or b = 3	Left operand is unambiguous, Right is not
=	b or a = b	3	Right operand is unambiguous, Left is not

Figure 1.11

Since both = have R to L associativity and only the second = has unambiguous right operand (necessary condition for R to L associativity) the second = is performed earlier.

Consider yet another expression

$z = a * b + c / d ;$

Here * and / enjoys same priority and same associativity (Left to Right). Figure 1.12 shows for each operator which operand is unambiguous and which is not.

Operator	Left	Right	Remark
*	a	b	Both operands are unambiguous
/	c	d	Both operands are unambiguous

Figure 1.12

Here since left operands for both operators are unambiguous Compiler is free to perform * or / operation as per its convenience

since no matter which is performed earlier the result would be same.

Appendix A gives the associativity of all the operators available in C.

Control Instructions in C

As the name suggests the ‘Control Instructions’ enable us to specify the order in which the various instructions in a program are to be executed by the computer. In other words the control instructions determine the ‘flow of control’ in a program. There are four types of control instructions in C. They are:

- (a) Sequence Control Instruction
- (b) Selection or Decision Control Instruction
- (c) Repetition or Loop Control Instruction
- (d) Case Control Instruction

The Sequence control instruction ensures that the instructions are executed in the same order in which they appear in the program. Decision and Case control instructions allow the computer to take a decision as to which instruction is to be executed next. The Loop control instruction helps computer to execute a group of statements repeatedly. In the following chapters we are going to learn these instructions in detail. Try your hand at the Exercise presented on the following pages before proceeding to the next chapter, which discusses the decision control instruction.

Summary

- (a) The three primary constants and variable types in C are integer, float and character.
- (b) A variable name can be of maximum 31 characters.
- (c) Do not use a keyword as a variable name.

- (d) An expression may contain any sequence of constants, variables and operators.
- (e) Operators having equal precedence are evaluated using associativity.
- (f) Left to right associativity means that the left operand of a operator must be unambiguous whereas right to left associativity means that the right operand of a operator must be unambiguous.
- (g) Input/output in C can be achieved using **scanf()** and **printf()** functions.

Exercise

[A] Which of the following are invalid variable names and why?

BASICSALARY	_basic	basic-hra
#MEAN	group.	422
population in 2006	over time	mindovermatter
FLOAT	hELLO	queue.
team'svictory	Plot # 3	2015_DDay

[B] Point out the errors, if any, in the following C statements:

- (a) `int = 314.562 * 150 ;`
- (b) `name = 'Ajay' ;`
- (c) `varchar = '3' ;`
- (d) `3.14 * r * r * h = vol_of_cyl ;`
- (e) `k = ( a * b ) ( c + ( 2.5a + b ) ( d + e ) ;`
- (f) `m_inst = rate of interest * amount in rs ;`

(g) $si = \text{principal} * \text{rateofinterest} * \text{numberofyears} / 100 ;$

(h) $\text{area} = 3.14 * r ** 2 ;$

(i) $\text{volume} = 3.14 * r ^ 2 * h ;$

(j) $k = ((a * b) + c) (2.5 * a + b) ;$

(k) $a = b = 3 = 4 ;$

(l) $\text{count} = \text{count} + 1 ;$

(m) $\text{date} = \text{'2 Mar 04'} ;$

[C] Evaluate the following expressions and show their hierarchy.

(a) $g = \text{big} / 2 + \text{big} * 4 / \text{big} - \text{big} + \text{abc} / 3 ;$
($\text{abc} = 2.5$, $\text{big} = 2$, assume **g** to be a float)

(b) $\text{on} = \text{ink} * \text{act} / 2 + 3 / 2 * \text{act} + 2 + \text{tig} ;$
($\text{ink} = 4$, $\text{act} = 1$, $\text{tig} = 3.2$, assume **on** to be an int)

(c) $s = \text{qui} * \text{add} / 4 - 6 / 2 + 2 / 3 * 6 / \text{god} ;$
($\text{qui} = 4$, $\text{add} = 2$, $\text{god} = 2$, assume **s** to be an int)

(d) $s = 1 / 3 * a / 4 - 6 / 2 + 2 / 3 * 6 / g ;$
($a = 4$, $g = 3$, assume **s** to be an int)

[D] Fill the following table for the expressions given below and then evaluate the result. A sample entry has been filled in the table for expression (a).

Operator	Left	Right	Remark
/	10	5 or 5 / 2 / 1	Left operand is unambiguous, Right is not
..	..	..	..

- (a) $g = 10 / 5 / 2 / 1 ;$
 (b) $b = 3 / 2 + 5 * 4 / 3 ;$
 (c) $a = b = c = 3 + 4 ;$

[E] Convert the following equations into corresponding C statements.

(a)
$$Z = \frac{8.8(a+b)^2/c - 0.5 + 2a/(q+r)}{(a+b)*(1/m)}$$

(b)
$$X = \frac{-b + (b*b) + 2 \cdot 4ac}{2a}$$

(c)
$$R = \frac{2v + 6.22(c+d)}{g+v}$$

(d)
$$A = \frac{7.7b(xy+a)/c - 0.8 + 2b}{(x+a)(1/y)}$$

[F] What would be the output of the following programs:

(a)

```
main()
{
```

```
int i = 2, j = 3, k, l ;
float a, b ;
k = i / j * j ;
l = j / i * i ;
a = i / j * j ;
b = j / i * i ;
printf( "%d %d %f %f", k, l, a, b ) ;
}
```

```
(b) main( )
{
    int a, b ;
    a = -3 - - 3 ;
    b = -3 - - ( - 3 ) ;
    printf ( "a = %d b = %d", a, b ) ;
}
```

```
(c) main( )
{
    float a = 5, b = 2 ;
    int c ;
    c = a % b ;
    printf ( "%d", c ) ;
}
```

```
(d) main( )
{
    printf ( "nn \n\n nn\n" ) ;
    printf ( "nn /n/n nn/n" ) ;
}
```

```
(e) main( )
{
    int a, b ;
    printf ( "Enter values of a and b" ) ;
    scanf ( " %d %d ", &a, &b ) ;
    printf ( "a = %d b = %d", a, b ) ;
}
```



```
(f)  main()  
    {  
        int p, q ;  
        printf ( "Enter values of p and q" ) ;  
        scanf ( " %d %d ", p, q ) ;  
        printf ( "p = %d q =%d", p, q ) ;  
    }
```

[G] Pick up the correct alternative for each of the following questions:

- (a) C language has been developed by
 - (1) Ken Thompson
 - (2) Dennis Ritchie
 - (3) Peter Norton
 - (4) Martin Richards
- (b) C can be used on
 - (1) Only MS-DOS operating system
 - (2) Only Linux operating system
 - (3) Only Windows operating system
 - (4) All the above
- (c) C programs are converted into machine language with the help of
 - (1) An Editor
 - (2) A compiler
 - (3) An operating system
 - (4) None of the above
- (d) The real constant in C can be expressed in which of the following forms
 - (1) Fractional form only
 - (2) Exponential form only
 - (3) ASCII form only

- (4) Both fractional and exponential forms
- (e) A character variable can at a time store
 - (1) 1 character
 - (2) 8 characters
 - (3) 254 characters
 - (4) None of the above
- (f) The statement **char ch = 'Z'** would store in **ch**
 - (1) The character Z
 - (2) ASCII value of Z
 - (3) Z along with the single inverted commas
 - (4) Both (1) and (2)
- (g) Which of the following is NOT a character constant
 - (1) 'Thank You'
 - (2) 'Enter values of P, N, R'
 - (3) '23.56E-03'
 - (4) All the above
- (h) The maximum value that an integer constant can have is
 - (1) -32767
 - (2) 32767
 - (3) 1.7014e+38
 - (4) -1.7014e+38
- (i) A C variable cannot start with
 - (1) An alphabet
 - (2) A number
 - (3) A special symbol other than underscore
 - (4) Both (2) & (3) above
- (j) Which of the following statement is wrong
 - (1) `mes = 123.56 ;`
 - (2) `con = 'T' * 'A' ;`
 - (3) `this = 'T' * 20 ;`
 - (4) `3 + a = b ;`

- (k) Which of the following shows the correct hierarchy of arithmetic operators in C
- (1) **, * or /, + or -
 - (2) **, *, /, +, -
 - (3) **, /, *, +, -
 - (4) / or *, - or +
- (l) In $b = 6.6 / a + 2 * n$; which operation will be performed first?
- (1) $6.6 / a$
 - (2) $a + 2$
 - (3) $2 * n$
 - (4) Depends upon compiler
- (m) Which of the following is allowed in a C Arithmetic instruction
- (1) []
 - (2) { }
 - (3) ()
 - (4) None of the above
- (n) Which of the following statements is false
- (1) Each new C instruction has to be written on a separate line
 - (2) Usually all C statements are entered in small case letters
 - (3) Blank spaces may be inserted between two words in a C statement
 - (4) Blank spaces cannot be inserted within a variable name
- (o) If a is an integer variable, $a = 5 / 2$; will return a value
- (1) 2.5
 - (2) 3
 - (3) 2
 - (4) 0
- (p) The expression, $a = 7 / 22 * (3.14 + 2) * 3 / 5$; evaluates to

- (1) 8.28
 - (2) 6.28
 - (3) 3.14
 - (4) 0
- (q) The expression, $a = 30 * 1000 + 2768$; evaluates to
- (1) 32768
 - (2) -32768
 - (3) 113040
 - (4) 0
- (r) The expression $x = 4 + 2 \% - 8$ evaluates to
- (1) -6
 - (2) 6
 - (3) 4
 - (4) None of the above
- (s) Hierarchy decides which operator
- (1) is most important
 - (2) is used first
 - (3) is fastest
 - (4) operates on largest numbers
- (t) An integer constant in C must have:
- (1) At least one digit
 - (2) Atleast one decimal point
 - (3) A comma along with digits
 - (4) Digits separated by commas
- (u) A character variable can never store more than
- (1) 32 characters
 - (2) 8 characters
 - (3) 254 characters
 - (4) 1 character
- (v) In C a variable cannot contain
- (1) Blank spaces

- (2) Hyphen
 - (3) Decimal point
 - (4) All the above
- (w) Which of the following is FALSE in C
- (1) Keywords can be used as variable names
 - (2) Variable names can contain a digit
 - (3) Variable names do not contain a blank space
 - (4) Capital letters can be used in variable names
- (x) In C, Arithmetic instruction cannot contain
- (1) variables
 - (2) constants
 - (3) variable names on right side of =
 - (4) constants on left side of =
- (y) Which of the following shows the correct hierarchy of arithmetic operations in C
- (1) / + * -
 - (2) * - / +
 - (3) + - / *
 - (4) * / + -
- (z) What will be the value of **d** if **d** is a float after the operation **d = 2 / 7.0**?
- (1) 0
 - (2) 0.2857
 - (3) Cannot be determined
 - (4) None of the above
- [H] Write C programs for the following:
- (a) Ramesh's basic salary is input through the keyboard. His dearness allowance is 40% of basic salary, and house rent allowance is 20% of basic salary. Write a program to calculate his gross salary.

- (b) The distance between two cities (in km.) is input through the keyboard. Write a program to convert and print this distance in meters, feet, inches and centimeters.
- (c) If the marks obtained by a student in five different subjects are input through the keyboard, find out the aggregate marks and percentage marks obtained by the student. Assume that the maximum marks that can be obtained by a student in each subject is 100.
- (d) Temperature of a city in Fahrenheit degrees is input through the keyboard. Write a program to convert this temperature into Centigrade degrees.
- (e) The length & breadth of a rectangle and radius of a circle are input through the keyboard. Write a program to calculate the area & perimeter of the rectangle, and the area & circumference of the circle.
- (f) Two numbers are input through the keyboard into two locations C and D. Write a program to interchange the contents of C and D.
- (g) If a five-digit number is input through the keyboard, write a program to calculate the sum of its digits.
(Hint: Use the modulus operator ‘%’)
- (h) If a five-digit number is input through the keyboard, write a program to reverse the number.
- (i) If a four-digit number is input through the keyboard, write a program to obtain the sum of the first and last digit of this number.
- (j) In a town, the percentage of men is 52. The percentage of total literacy is 48. If total percentage of literate men is 35 of the total population, write a program to find the total number

of illiterate men and women if the population of the town is 80,000.

- (k) A cashier has currency notes of denominations 10, 50 and 100. If the amount to be withdrawn is input through the keyboard in hundreds, find the total number of currency notes of each denomination the cashier will have to give to the withdrawer.
- (l) If the total selling price of 15 items and the total profit earned on them is input through the keyboard, write a program to find the cost price of one item.
- (m) If a five-digit number is input through the keyboard, write a program to print a new number by adding one to each of its digits. For example if the number that is input is 12391 then the output should be displayed as 23402.

2 *The Decision Control Structure*

- Decisions! Decisions!
- The *if* Statement
 - The Real Thing
 - Multiple Statements within *if*
- The *if-else* Statement
 - Nested *if-elses*
 - Forms of *if*
- Use of Logical Operators
 - The *else if* Clause
 - The ! Operator
 - Hierarchy of Operators Revisited
- A Word of Caution
- The Conditional Operators
- Summary
- Exercise

We all need to alter our actions in the face of changing circumstances. If the weather is fine, then I will go for a stroll. If the highway is busy I would take a diversion. If the pitch takes spin, we would win the match. If she says no, I would look elsewhere. If you like this book, I would write the next edition. You can notice that all these decisions depend on some condition being met.

C language too must be able to perform different sets of actions depending on the circumstances. In fact this is what makes it worth its salt. C has three major decision making instructions—the **if** statement, the **if-else** statement, and the **switch** statement. A fourth, somewhat less important structure is the one that uses conditional operators. In this chapter we will explore all these ways (except **switch**, which has a separate chapter devoted to it, later) in which a C program can react to changing circumstances.

Decisions! Decisions!

In the programs written in Chapter 1 we have used sequence control structure in which the various steps are executed sequentially, i.e. in the same order in which they appear in the program. In fact to execute the instructions sequentially, we don't have to do anything at all. By default the instructions in a program are executed sequentially. However, in serious programming situations, seldom do we want the instructions to be executed sequentially. Many a times, we want a set of instructions to be executed in one situation, and an entirely different set of instructions to be executed in another situation. This kind of situation is dealt in C programs using a decision control instruction. As mentioned earlier, a decision control instruction can be implemented in C using:

- (a) The **if** statement
- (b) The **if-else** statement
- (c) The conditional operators

Now let us learn each of these and their variations in turn.

The *if* Statement

Like most languages, C uses the keyword **if** to implement the decision control instruction. The general form of **if** statement looks like this:

```
if ( this condition is true )  
    execute this statement ;
```

The keyword **if** tells the compiler that what follows is a decision control instruction. The condition following the keyword **if** is always enclosed within a pair of parentheses. If the condition, whatever it is, is true, then the statement is executed. If the condition is not true then the statement is not executed; instead the program skips past it. But how do we express the condition itself in C? And how do we evaluate its truth or falsity? As a general rule, we express a condition using C’s ‘relational’ operators. The relational operators allow us to compare two values to see whether they are equal to each other, unequal, or whether one is greater than the other. Here’s how they look and how they are evaluated in C.

this expression	is true if
x == y	x is equal to y
x != y	x is not equal to y
x < y	x is less than y
x > y	x is greater than y
x <= y	x is less than or equal to y
x >= y	x is greater than or equal to y

Figure 2.1

The relational operators should be familiar to you except for the equality operator `==` and the inequality operator `!=`. Note that `=` is used for assignment, whereas, `==` is used for comparison of two quantities. Here is a simple program, which demonstrates the use of **if** and the relational operators.

```
/* Demonstration of if statement */
main()
{
    int  num ;

    printf ( "Enter a number less than 10 " ) ;
    scanf ( "%d", &num ) ;

    if ( num <= 10 )
        printf ( "What an obedient servant you are !" ) ;
}
```

On execution of this program, if you type a number less than or equal to 10, you get a message on the screen through **printf()**. If you type some other number the program doesn't do anything. The following flowchart would help you understand the flow of control in the program.

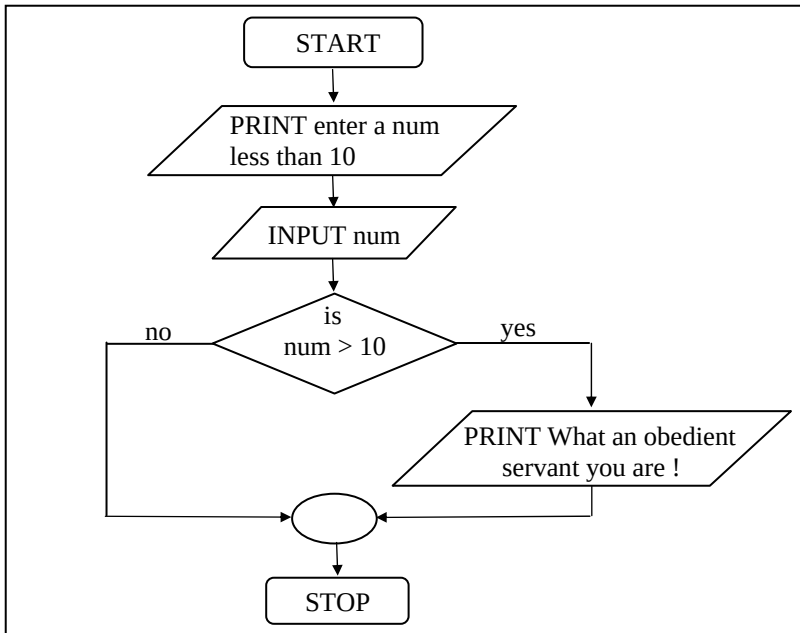


Figure 2.2

To make you comfortable with the decision control instruction one more example has been given below. Study it carefully before reading further. To help you understand it easily, the program is accompanied by an appropriate flowchart.

Example 2.1: While purchasing certain items, a discount of 10% is offered if the quantity purchased is more than 1000. If quantity and price per item are input through the keyboard, write a program to calculate the total expenses.

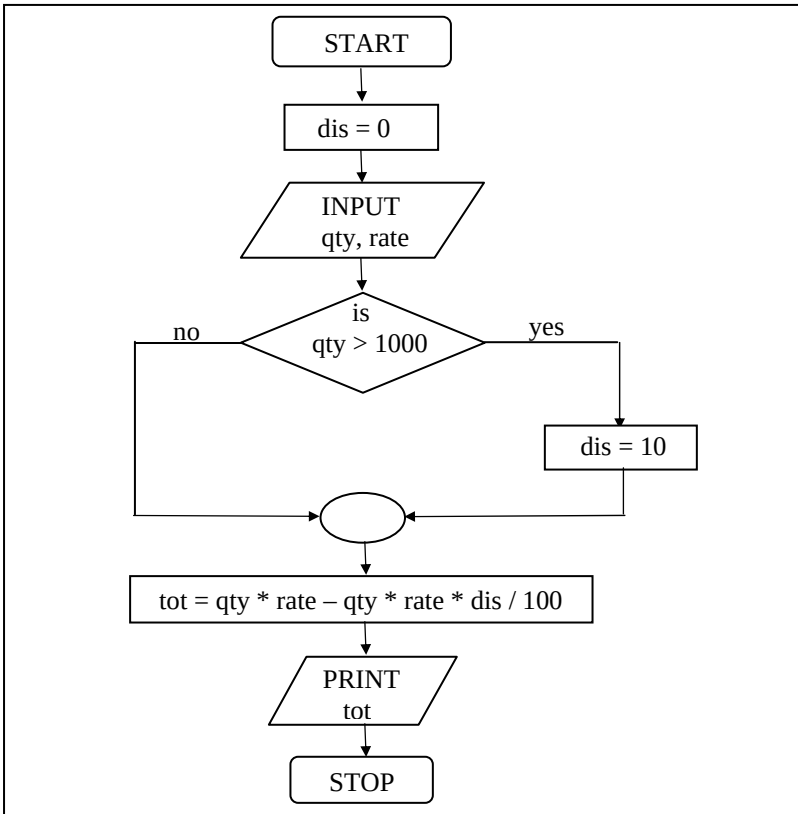


Figure 2.3

```
/* Calculation of total expenses */
main()
{
    int  qty, dis = 0 ;
    float rate, tot ;
    printf ( "Enter quantity and rate " ) ;
    scanf ( "%d %f", &qty, &rate) ;

    if ( qty > 1000 )
        dis = 10 ;
```

```
tot = ( qty * rate ) - ( qty * rate * dis / 100 ) ;  
printf ( "Total expenses = Rs. %f", tot ) ;  
}
```

Here is some sample interaction with the program.

```
Enter quantity and rate 1200 15.50  
Total expenses = Rs. 16740.000000
```

```
Enter quantity and rate 200 15.50  
Total expenses = Rs. 3100.000000
```

In the first run of the program, the condition evaluates to true, as 1200 (value of **qty**) is greater than 1000. Therefore, the variable **dis**, which was earlier set to 0, now gets a new value 10. Using this new value total expenses are calculated and printed.

In the second run the condition evaluates to false, as 200 (the value of **qty**) isn't greater than 1000. Thus, **dis**, which is earlier set to 0, remains 0, and hence the expression after the minus sign evaluates to zero, thereby offering no discount.

Is the statement **dis = 0** necessary? The answer is yes, since in C, a variable if not specifically initialized contains some unpredictable value (garbage value).

The Real Thing

We mentioned earlier that the general form of the if statement is as follows

```
if ( condition )  
    statement ;
```

Truly speaking the general form is as follows:

```
if ( expression )  
    statement ;
```

Here the expression can be any valid expression including a relational expression. We can even use arithmetic expressions in the **if** statement. For example all the following **if** statements are valid

```
if ( 3 + 2 % 5 )  
    printf ( "This works" ) ;
```

```
if ( a = 10 )  
    printf ( "Even this works" ) ;
```

```
if ( -5 )  
    printf ( "Surprisingly even this works" ) ;
```

Note that in C a non-zero value is considered to be true, whereas a 0 is considered to be false. In the first **if**, the expression evaluates to 5 and since 5 is non-zero it is considered to be true. Hence the **printf()** gets executed.

In the second **if**, 10 gets assigned to **a** so the **if** is now reduced to **if (a)** or **if (10)**. Since 10 is non-zero, it is true hence again **printf()** goes to work.

In the third **if**, -5 is a non-zero number, hence true. So again **printf()** goes to work. In place of -5 even if a float like 3.14 were used it would be considered to be true. So the issue is not whether the number is integer or float, or whether it is positive or negative. Issue is whether it is zero or non-zero.

Multiple Statements within *if*

It may so happen that in a program we want more than one statement to be executed if the expression following **if** is satisfied. If such multiple statements are to be executed then they must be

placed within a pair of braces as illustrated in the following example.

Example 2.2: The current year and the year in which the employee joined the organization are entered through the keyboard. If the number of years for which the employee has served the organization is greater than 3 then a bonus of Rs. 2500/- is given to the employee. If the years of service are not greater than 3, then the program should do nothing.

```
/* Calculation of bonus */
main()
{
    int  bonus, cy, yoj, yr_of_ser ;

    printf ( "Enter current year and year of joining " ) ;
    scanf ( "%d %d", &cy, &yoy ) ;

    yr_of_ser = cy - yoj ;

    if ( yr_of_ser > 3 )
    {
        bonus = 2500 ;
        printf ( "Bonus = Rs. %d", bonus ) ;
    }
}
```

Observe that here the two statements to be executed on satisfaction of the condition have been enclosed within a pair of braces. If a pair of braces is not used then the C compiler assumes that the programmer wants only the immediately next statement after the **if** to be executed on satisfaction of the condition. In other words we can say that the default scope of the **if** statement is the immediately next statement after it.

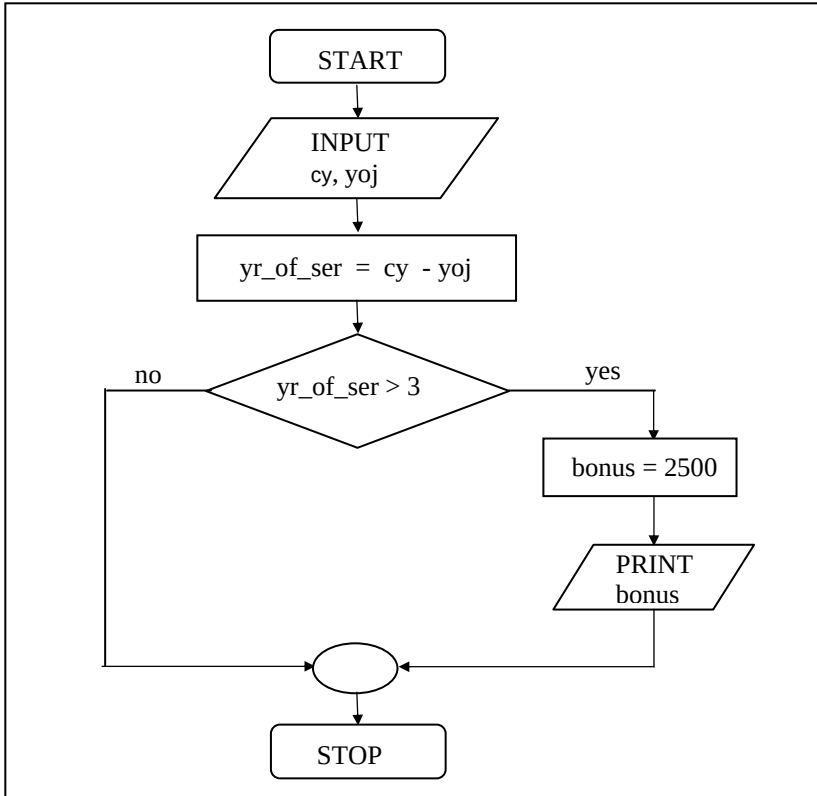


Figure 2.4

The *if-else* Statement

The **if** statement by itself will execute a single statement, or a group of statements, when the expression following **if** evaluates to true. It does nothing when the expression evaluates to false. Can we execute one group of statements if the expression evaluates to true and another group of statements if the expression evaluates to false? Of course! This is what is the purpose of the **else** statement that is demonstrated in the following example:

Example 2.3: In a company an employee is paid as under:

If his basic salary is less than Rs. 1500, then HRA = 10% of basic salary and DA = 90% of basic salary. If his salary is either equal to or above Rs. 1500, then HRA = Rs. 500 and DA = 98% of basic salary. If the employee's salary is input through the keyboard write a program to find his gross salary.

```
/* Calculation of gross salary */
main()
{
    float bs, gs, da, hra ;

    printf ( "Enter basic salary " ) ;
    scanf ( "%f", &bs ) ;

    if ( bs < 1500 )
    {
        hra = bs * 10 / 100 ;
        da = bs * 90 / 100 ;
    }
    else
    {
        hra = 500 ;
        da = bs * 98 / 100 ;
    }

    gs = bs + hra + da ;
    printf ( "gross salary = Rs. %f", gs ) ;
}
```

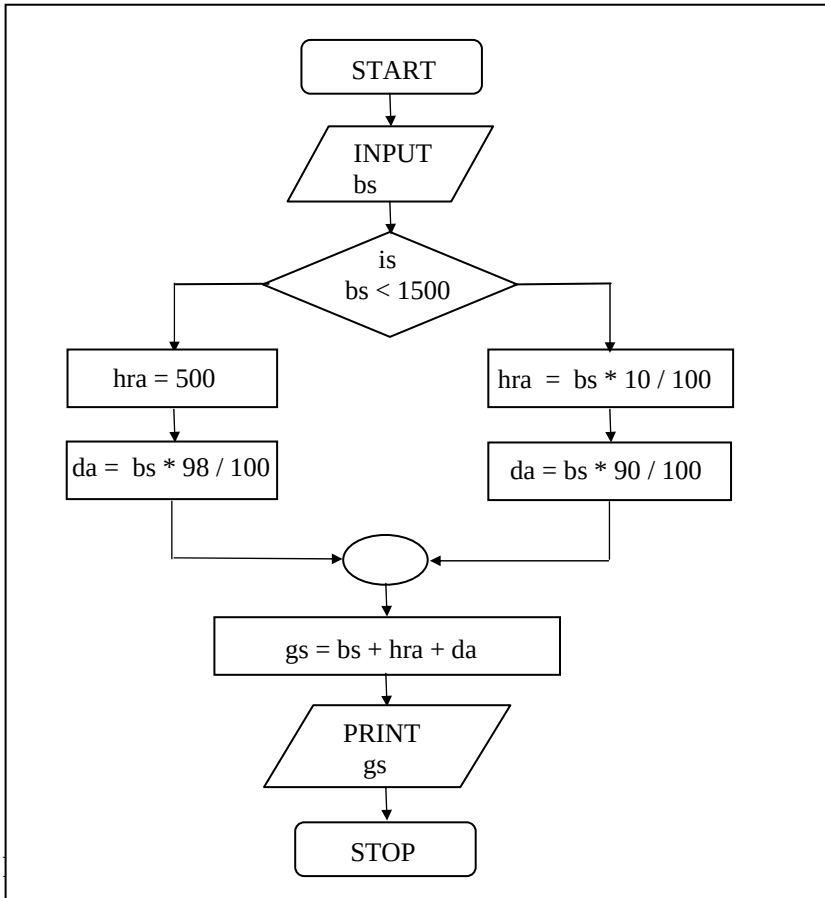


Figure 2.5

A few points worth noting...

- The group of statements after the **if** upto and not including the **else** is called an 'if block'. Similarly, the statements after the **else** form the 'else block'.
- Notice that the **else** is written exactly below the **if**. The statements in the if block and those in the else block have been indented to the right. This formatting convention is

followed throughout the book to enable you to understand the working of the program better.

- (c) Had there been only one statement to be executed in the **if** block and only one statement in the **else** block we could have dropped the pair of braces.
- (d) As with the **if** statement, the default scope of **else** is also the statement immediately after the **else**. To override this default scope a pair of braces as shown in the above example must be used.

Nested *if-elses*

It is perfectly all right if we write an entire **if-else** construct within either the body of the **if** statement or the body of an **else** statement. This is called ‘nesting’ of **ifs**. This is shown in the following program.

```
/* A quick demo of nested if-else */
main()
{
    int i;

    printf ( "Enter either 1 or 2 " );
    scanf ( "%d", &i );

    if ( i == 1 )
        printf ( "You would go to heaven !" );
    else
    {
        if ( i == 2 )
            printf ( "Hell was created with you in mind" );
        else
            printf ( "How about mother earth !" );
    }
}
```

Note that the second **if-else** construct is nested in the first **else** statement. If the condition in the first **if** statement is false, then the condition in the second **if** statement is checked. If it is false as well, then the final **else** statement is executed.

You can see in the program how each time a **if-else** construct is nested within another **if-else** construct, it is also indented to add clarity to the program. Inculcate this habit of indentation, otherwise you would end up writing programs which nobody (you included) can understand easily at a later date.

In the above program an **if-else** occurs within the **else** block of the first **if** statement. Similarly, in some other program an **if-else** may occur in the **if** block as well. There is no limit on how deeply the **ifs** and the **elses** can be nested.

Forms of *if*

The **if** statement can take any of the following forms:

- (a) `if ( condition )`
 `do this ;`
- (b) `if ( condition )`
 `{`
 `do this ;`
 `and this ;`
 `}`
- (c) `if ( condition )`
 `do this ;`
 `else`
 `do this ;`
- (d) `if ( condition )`
 `{`
 `do this ;`

```
        and this ;  
    }  
    else  
    {  
        do this ;  
        and this ;  
    }
```

```
(e)  if ( condition )  
        do this ;  
    else  
    {  
        if ( condition )  
            do this ;  
        else  
        {  
            do this ;  
            and this ;  
        }  
    }
```

```
(f)  if ( condition )  
    {  
        if ( condition )  
            do this ;  
        else  
        {  
            do this ;  
            and this ;  
        }  
    }  
    else  
        do this ;
```

Use of Logical Operators

C allows usage of three logical operators, namely, `&&`, `||` and `!`. These are to be read as ‘AND’ ‘OR’ and ‘NOT’ respectively.

There are several things to note about these logical operators. Most obviously, two of them are composed of double symbols: `||` and `&&`. Don't use the single symbol `|` and `&`. These single symbols also have a meaning. They are bitwise operators, which we would examine in Chapter 14.

The first two operators, `&&` and `||`, allow two or more conditions to be combined in an **if** statement. Let us see how they are used in a program. Consider the following example.

Example 2.4: The marks obtained by a student in 5 different subjects are input through the keyboard. The student gets a division as per the following rules:

Percentage above or equal to 60 - First division
Percentage between 50 and 59 - Second division
Percentage between 40 and 49 - Third division
Percentage less than 40 - Fail

Write a program to calculate the division obtained by the student.

There are two ways in which we can write a program for this example. These methods are given below.

```
/* Method – I */
main()
{
    int m1, m2, m3, m4, m5, per ;

    printf ( "Enter marks in five subjects " ) ;
    scanf ( "%d %d %d %d %d", &m1, &m2, &m3, &m4, &m5 ) ;
    per = ( m1 + m2 + m3 + m4 + m5 ) / 5 ;
```

```
if ( per >= 60 )
    printf ( "First division " );
else
{
    if ( per >= 50 )
        printf ( "Second division" );
    else
    {
        if ( per >= 40 )
            printf ( "Third division" );
        else
            printf ( "Fail" );
    }
}
```

This is a straight forward program. Observe that the program uses nested **if-elses**. This leads to three disadvantages:

- (a) As the number of conditions go on increasing the level of indentation also goes on increasing. As a result the whole program creeps to the right.
- (b) Care needs to be exercised to match the corresponding **ifs** and **elses**.
- (c) Care needs to be exercised to match the corresponding pair of braces.

All these three problems can be eliminated by usage of ‘Logical operators’. The following program illustrates this.

```
/* Method – II */
main()
{
    int m1, m2, m3, m4, m5, per ;

    printf ( "Enter marks in five subjects " );
    scanf ( "%d %d %d %d %d", &m1, &m2, &m3, &m4, &m5 );
    per = ( m1 + m2 + m3 + m4 + m5 ) / 5 ;
}
```



```
if ( per >= 60 )
    printf ( "First division" );

if ( ( per >= 50 ) && ( per < 60 ) )
    printf ( "Second division" );

if ( ( per >= 40 ) && ( per < 50 ) )
    printf ( "Third division" );

if ( per < 40 )
    printf ( "Fail" );
}
```

As can be seen from the second **if** statement, the **&&** operator is used to combine two conditions. ‘Second division’ gets printed if both the conditions evaluate to true. If one of the conditions evaluate to false then the whole thing is treated as false.

Two distinct advantages can be cited in favour of this program:

- (a) The matching (or do I say mismatching) of the **ifs** with their corresponding **elses** gets avoided, since there are no **elses** in this program.
- (b) In spite of using several conditions, the program doesn't creep to the right. In the previous program the statements went on creeping to the right. This effect becomes more pronounced as the number of conditions go on increasing. This would make the task of matching the **ifs** with their corresponding **elses** and matching of opening and closing braces that much more difficult.

The *else if* Clause

There is one more way in which we can write program for Example 2.4. This involves usage of **else if** blocks as shown below:

```
/* else if ladder demo */
main()
{
    int  m1, m2, m3, m4, m5, per ;

    per = ( m1+ m2 + m3 + m4+ m5 ) / per ;

    if ( per >= 60 )
        printf ( "First division" ) ;
    else if ( per >= 50 )
        printf ( "Second division" ) ;
    else if ( per >= 40 )
        printf ( "Third division" ) ;
    else
        printf ( "fail" ) ;
}
```

You can note that this program reduces the indentation of the statements. In this case every **else** is associated with its previous **if**. The last **else** goes to work only if all the conditions fail. Even in **else if** ladder the last **else** is optional.

Note that the **else if** clause is nothing different. It is just a way of rearranging the **else** with the **if** that follows it. This would be evident if you look at the following code:

<pre>if (i == 2) printf ("With you...") ; else { if (j == 2) printf ("...All the time") ; }</pre>	<pre>if (i == 2) printf ("With you...") ; else if (j == 2) printf ("...All the time ") ;</pre>
---	--

Another place where logical operators are useful is when we want to write programs for complicated logics that ultimately boil down

to only two answers. For example, consider the following example:

Example 2.5: A company insures its drivers in the following cases:

- If the driver is married.
- If the driver is unmarried, male & above 30 years of age.
- If the driver is unmarried, female & above 25 years of age.

In all other cases the driver is not insured. If the marital status, sex and age of the driver are the inputs, write a program to determine whether the driver is to be insured or not.

Here after checking a complicated set of instructions the final output of the program would be one of the two—Either the driver should be ensured or the driver should not be ensured. As mentioned above, since these are the only two outcomes this problem can be solved using logical operators. But before we do that let us write a program that does not make use of logical operators.

```
/* Insurance of driver - without using logical operators */
main()
{
    char sex, ms ;
    int age ;

    printf ( "Enter age, sex, marital status " ) ;
    scanf ( "%d %c %c", &age, &sex, &ms ) ;

    if ( ms == 'M' )
        printf ( "Driver is insured" ) ;
    else
    {
        if ( sex == 'M' )
        {
```

```
        if ( age > 30 )
            printf ( "Driver is insured" ) ;
        else
            printf ( "Driver is not insured" ) ;
    }
else
{
    if ( age > 25 )
        printf ( "Driver is insured" ) ;
    else
        printf ( "Driver is not insured" ) ;
}
}
```

From the program it is evident that we are required to match several **ifs** and **elses** and several pairs of braces. In a more real-life situation there would be more conditions to check leading to the program creeping to the right. Let us now see how to avoid these problems by using logical operators.

As mentioned above, in this example we expect the answer to be either 'Driver is insured' or 'Driver is not insured'. If we list down all those cases in which the driver is insured, then they would be:

- (a) Driver is married.
- (b) Driver is an unmarried male above 30 years of age.
- (c) Driver is an unmarried female above 25 years of age.

Since all these cases lead to the driver being insured, they can be combined together using **&&** and **||** as shown in the program below:

```
/* Insurance of driver - using logical operators */
main()
{
    char sex, ms ;
```

```
int age ;

printf ( "Enter age, sex, marital status " ) ;
scanf ( "%d %c %c" &age, &sex, &ms ) ;

if ( ( ms == 'M' ) || ( ms == 'U' && sex == 'M' && age > 30 ) ||
      ( ms == 'U' && sex == 'F' && age > 25 ) )
    printf ( "Driver is insured" ) ;
else
    printf ( "Driver is not insured" ) ;
}
```

In this program it is important to note that:

- The driver will be insured only if one of the conditions enclosed in parentheses evaluates to true.
- For the second pair of parentheses to evaluate to true, each condition in the parentheses separated by **&&** must evaluate to true.
- Even if one of the conditions in the second parentheses evaluates to false, then the whole of the second parentheses evaluates to false.
- The last two of the above arguments apply to third pair of parentheses as well.

Thus we can conclude that the **&&** and **||** are useful in the following programming situations:

- (a) When it is to be tested whether a value falls within a particular range or not.
- (b) When after testing several conditions the outcome is only one of the two answers (This problem is often called yes/no problem).

There can be one more situation other than checking ranges or yes/no problem where you might find logical operators useful. The following program demonstrates it.

Example 2.6: Write a program to calculate the salary as per the following table:

Gender	Years of Service	Qualifications	Salary
Male	>= 10	Post-Graduate	15000
	>= 10	Graduate	10000
	< 10	Post-Graduate	10000
	< 10	Graduate	7000
Female	>= 10	Post-Graduate	12000
	>= 10	Graduate	9000
	< 10	Post-Graduate	10000
	< 10	Graduate	6000

Figure 2.6

```
main()
{
    char g ;
    int  yos, qual, sal ;

    printf ( "Enter Gender, Years of Service and
              Qualifications ( 0 = G, 1 = PG ):" );
    scanf ( "%c%d%d", &g, &yos, &qual );

    if ( g == 'm' && yos >= 10 && qual == 1 )
        sal = 15000 ;
    else if ( ( g == 'm' && yos >= 10 && qual == 0 ) ||
              ( g == 'm' && yos < 10 && qual == 1 ) )
        sal = 10000 ;
```

```
    else if ( g == 'm' && yos < 10 && qual == 0 )
        sal = 7000 ;
    else if ( g == 'f' && yos >= 10 && qual == 1 )
        sal = 12000 ;
    else if ( g == 'f' && yos >= 10 && qual == 0 )
        sal = 9000 ;
    else if ( g == 'f' && yos < 10 && qual == 1 )
        sal = 10000 ;
    else if ( g == 'f' && yos < 10 && qual == 0 )
        sal = 6000 ;

    printf ( "\nSalary of Employee = %d", sal ) ;
}
```

The ! Operator

So far we have used only the logical operators **&&** and **||**. The third logical operator is the NOT operator, written as **!**. This operator reverses the result of the expression it operates on. For example, if the expression evaluates to a non-zero value, then applying **!** operator to it results into a 0. Vice versa, if the expression evaluates to zero then on applying **!** operator to it makes it 1, a non-zero value. The final result (after applying **!**) 0 or 1 is considered to be false or true respectively. Here is an example of the NOT operator applied to a relational expression.

```
!( y < 10 )
```

This means “not **y** less than 10”. In other words, if **y** is less than 10, the expression will be false, since (**y < 10**) is true. We can express the same condition as (**y >= 10**).

The NOT operator is often used to reverse the logical value of a single variable, as in the expression

```
if ( ! flag )
```

This is another way of saying

```
if ( flag == 0 )
```

Does the NOT operator sound confusing? Avoid it if you want, as the same thing can be achieved without using the NOT operator.

Hierarchy of Operators Revisited

Since we have now added the logical operators to the list of operators we know, it is time to review these operators and their priorities. Figure 2.7 summarizes the operators we have seen so far. The higher the position of an operator is in the table, higher is its priority. (A full-fledged precedence table of operators is given in Appendix A.)

Operators	Type
!	Logical NOT
* / %	Arithmetic and modulus
+ -	Arithmetic
< > <= >=	Relational
== !=	Relational
&&	Logical AND
	Logical OR
=	Assignment

Figure 2.7

A Word of Caution

What will be the output of the following program:


```
main()
{
    int i;

    printf ( "Enter value of i " );
    scanf ( "%d", &i );
    if ( i = 5 )
        printf ( "You entered 5" );
    else
        printf ( "You entered something other than 5" );
}
```

And here is the output of two runs of this program...

```
Enter value of i 200
You entered 5
Enter value of i 9999
You entered 5
```

Surprising? You have entered 200 and 9999, and still you find in either case the output is ‘You entered 5’. This is because we have written the condition wrongly. We have used the assignment operator = instead of the relational operator ==. As a result, the condition gets reduced to **if (5)**, irrespective of what you supply as the value of **i**. And remember that in C ‘truth’ is always non-zero, whereas ‘falsity’ is always zero. Therefore, **if (5)** always evaluates to true and hence the result.

Another common mistake while using the **if** statement is to write a semicolon (;) after the condition, as shown below:

```
main()
{
    int i;

    printf ( "Enter value of i " );
    scanf ( "%d", &i );
```

```
    if ( i == 5 );  
        printf ( "You entered 5" );  
}
```

The ; makes the compiler to interpret the statement as if you have written it in following manner:

```
if ( i == 5 )  
;  
printf ( "You entered 5" );
```

Here, if the condition evaluates to true the ; (null statement, which does nothing on execution) gets executed, following which the **printf()** gets executed. If the condition fails then straightaway the **printf()** gets executed. Thus, irrespective of whether the condition evaluates to true or false the **printf()** is bound to get executed. Remember that the compiler would not point out this as an error, since as far as the syntax is concerned nothing has gone wrong, but the logic has certainly gone awry. Moral is, beware of such pitfalls.

The following figure summarizes the working of all the three logical operators.

Operands		Results			
x	y	!x	!y	x && y	x y
0	0	1	1	0	0
0	non-zero	1	0	0	0
non-zero	0	0	1	0	1
non-zero	non-zero	0	0	1	1

Figure 2.8

The Conditional Operators

The conditional operators **?** and **:** are sometimes called ternary operators since they take three arguments. In fact, they form a kind of foreshortened if-then-else. Their general form is,

expression 1 **?** expression 2 **:** expression 3

What this expression says is: “if **expression 1** is true (that is, if its value is non-zero), then the value returned will be **expression 2**, otherwise the value returned will be **expression 3**”. Let us understand this with the help of a few examples:

(a)

```
int x, y;
scanf ("%d", &x);
y = ( x > 5 ? 3 : 4 );
```

This statement will store 3 in **y** if **x** is greater than 5, otherwise it will store 4 in **y**.

The equivalent **if** statement will be,

```
if ( x > 5 )
    y = 3 ;
else
    y = 4 ;
```

(b)

```
char a ;
int y ;
scanf ("%c", &a );
y = ( a >= 65 && a <= 90 ? 1 : 0 );
```

Here 1 would be assigned to **y** if **a >=65 && a <=90** evaluates to true, otherwise 0 would be assigned.

The following points may be noted about the conditional operators:

- (a) It's not necessary that the conditional operators should be used only in arithmetic statements. This is illustrated in the following examples:

```
Ex.:  int i;
      scanf ( "%d", &i );
      ( i == 1 ? printf ( "Amit" ) : printf ( "All and sundry" ) );
```

```
Ex.:  char a = 'z';
      printf ( "%c", ( a >= 'a' ? a : '!' ) );
```

- (b) The conditional operators can be nested as shown below.

```
int big, a, b, c;
big = ( a > b ? ( a > c ? 3 : 4 ) : ( b > c ? 6 : 8 ) );
```

- (c) Check out the following conditional expression:

```
a > b ? g = a : g = b ;
```

This will give you an error 'Lvalue Required'. The error can be overcome by enclosing the statement in the `:` part within a pair of parenthesis. This is shown below:

```
a > b ? g = a : ( g = b );
```

In absence of parentheses the compiler believes that **b** is being assigned to the result of the expression to the left of second `=`. Hence it reports an error.

The limitation of the conditional operators is that after the `?` or after the `:` only one C statement can occur. In practice rarely is this the requirement. Therefore, in serious C programming conditional operators aren't as frequently used as the **if-else**.

Summary

- (a) There are three ways for taking decisions in a program. First way is to use the **if-else** statement, second way is to use the